

AskMoses.AI — System Manual

How Every Number Is Calculated

Reference guide for answering rule questions and telling intended behaviour apart from real defects

Version 1.3 · September 2026 Supersedes v1.2 (September 2026). v1.1 was verified against `dev @ aafcf10` . **v1.3 updates §2.6, §9 and §16, re-verified against `dev @ 66cf2c6` (30 September 2026).** v1.2 updated §2.1, §2.4, §2.5, §2.6, §9, §11.2, §11.3, §15, §16 and §17, verified against `18807ca` . Everything else is still as verified at `aafcf10` . Every claim below carries a `file:line` reference so it can be re-checked.

Table of Contents

1. [How to Use This Manual](#)
 2. [Core Concepts](#)
 3. [The Owner Dashboard](#)
 4. [Analytics](#)
 5. [Script Intelligence \(Insights\)](#)
 6. [Script Gap Detection](#)
 7. [History](#)
 8. [The Rep's Dashboard \(My Page\)](#)
 9. [Calls and Call Detail](#)
 10. [Team Command Center](#)
 11. [Intent Analysis](#)
 12. [Appointments and Call Date](#)
 13. [Marketing Intelligence](#)
 14. [Admin Panel and Billing](#)
 15. [Fixed Since v1.0](#)
 16. [Known Defects — Do Not Report These](#)
 17. [Quick Reference](#)
-

1. How to Use This Manual

This manual exists so that when someone says "this number looks wrong", you can check here first and answer with confidence — is it working as designed, is it a known limitation, or is it genuinely broken?

Every metric in this document is described in three parts: what it is, how it is calculated, and why it works that way. The "why" matters most: almost every number that looks strange at first glance is the result of a deliberate trade-off, and knowing the trade-off is what lets you answer a prospect or a teammate without escalating to engineering.

The four callout boxes

WHY THIS EXISTS

The reasoning behind a design decision. If someone asks "why does it do that?", the answer is here.

OFTEN MISTAKEN FOR A BUG

Behaviour that looks like a bug but is intentional. If a report matches one of these, it is not a defect — explain the reasoning instead.

KNOWN DEFECT — ALREADY REPORTED

A genuine defect that has already been identified and verified against the code. No need to report it again; it is on the fix list. Section 16 collects all of them in one place.

FIXED SINCE v1.0

Something the previous edition of this manual described as broken, which has since been repaired. Section 15 collects these. If you are working from the old PDF, these are the entries to unlearn.

Trust markers on data

Not every number in the product has the same standing. Some are arithmetic on real calls; some are the AI's opinion; some are placeholder values left over from the demo phase.

Marker	Meaning
CALCULATE D	Arithmetic performed on real call data. Reproducible and auditable.
AI OPINION	A number the language model was asked to produce. It looks like a statistic but no code verifies it. Treat it as a qualified guess, not a measurement.
PLACEHOLD ER	A fixed value hardcoded during the demo phase. It does not come from data at all and will not change no matter what the client does.

This distinction is the single most useful thing in this manual. The most common category of "false bug" is someone noticing that an **AI OPINION** number does not reconcile with a **CALCULATED** one. They are not supposed to reconcile — they are different kinds of claim.

2. Core Concepts

2.1 How a call gets scored

Every call, regardless of how it enters the system, follows the same path:

1. **Audio arrives** — either uploaded manually or pulled automatically from the CRM.
2. **Whisper transcribes** it into text.
3. **GPT reads the transcript** against the organisation's active script (the rubric).
4. The AI returns, in one structured response: a score from **0 to 100** for each section, a written justification per section, strengths, improvements, a summary, and its reading of how the call ended.
5. The overall score is computed from those section scores.

The five sections are Discovery, Problem Agitation, Offer Presentation, Objection Handling, and Close & Next Steps (defaults at `lib/services/scoring.ts:120–124`).

CONVENTION — NOT ENFORCED BY THE SYSTEM

Every client script must have **exactly these five sections, with these names, in this order**, and no section may be left empty. This is an operating rule (User Guide §7), not something the product checks: the database has no constraint on `scripts.sections` (no CHECK, no trigger), `POST /api/scripts` stores whatever it receives (`app/api/scripts/route.ts:64–76`), and the Rubric Config form lets you rename sections and add more with **Add Section**. A script that breaks the convention is accepted and scored as-is — an empty section scores low and drags down every call.

The overall score is a weighted average

$$\text{overall_score} = \sum (\text{section score} \times \text{section weight}) \div \sum \text{weights}, \text{ rounded to an integer}$$

Falls back to a **plain average** when any section has no weight, or when the weights add up to 0.

`lib/services/overall-score.ts` — `computeOverallScore()` , used by both scoring paths
(`lib/services/scoring.ts:538` , `app/api/analyze/route.ts:670`)

Weights are looked up by section name (case-insensitive), using the names configured in the script. A section the AI returns under a name the script has no weight for sends the whole call to the plain average.

WHY THIS EXISTS

The score measures **execution quality**, not results. A rep can run a textbook call and still lose the deal to a prospect with no budget; another can stumble through a call with a customer who was always going to buy. Keeping the score independent of the outcome is what makes it useful for coaching — it answers "how well did you sell?", not "did you win?". The outcome is tracked separately, as its own field.

OFTEN MISTAKEN FOR A BUG

The score is **not capped by the outcome**. A call that did not close can still score 95, and a call that closed can score 40. This is deliberate and reps notice it. It is not a bug — it is the whole point of separating quality from results.

2.2 The two scales: 0–100 internally, 0–5 on screen

The AI is asked to score on a 0–100 scale, and 0–100 is what the database stores. Every screen that shows a score divides by 20 and displays it with one decimal.

```
displayed score = stored score ÷ 20
```

```
84 / 100 → shown as 4.2 / 5 72 / 100 → shown as 3.6 / 5
```

```
lib/score-display.ts — toDisplay5(), toNumber5()
```

WHY THIS EXISTS

The AI produces better, more consistent judgements when it has a wide range to work with — the gap between an 82 and an 87 is a distinction it can reason about, whereas forcing it to pick between 4 and 5 loses that nuance. But reps and owners read a 0–5 rating far more naturally. So the system **reasons in 100 and speaks in 5**.

The scoring bands the AI is given are: 90–100 textbook, 75–89 strong, 60–74 adequate, 40–59 needs work, 0–39 poor or absent.

OFTEN MISTAKEN FOR A BUG

If someone reports "the score in the database doesn't match the screen", check whether they are comparing 0–100 to 0–5. A database value of 84 showing as "4.2" on screen is correct behaviour, not data corruption.

2.3 Colour bands and thresholds

Band	Stored (0–100)	Displayed (0–5)	Meaning
 Green	85 and above	4.25 and above	Strong execution
 Amber	70 to 84	3.5 to 4.2	Adequate, room to improve
 Red	below 70	below 3.5	Needs coaching attention

```
lib/score-display.ts — scoreLevel() . A separate threshold, 95 and above  
( PERFECT_CALL_THRESHOLD ), marks a call as a "Perfect Call" in the Analytics achievements.
```

OFTEN MISTAKEN FOR A BUG

One screen uses slightly different cut-offs. On the call detail page, the written feedback fallback text switches tiers at 80 and 60, not 85 and 70. This means a call scoring 82 shows an amber bar next to more complimentary text. It is a minor inconsistency that exists because the two systems were built for different purposes — but it is not data corruption.

NOTE ON `lib/score-display.ts`

This module was introduced as the single source of truth for scale conversion and thresholds. Its header states that no other file should perform `/ 20`, `* 20`, or inline threshold comparisons. When adding code that touches scores, add a helper there rather than inlining the arithmetic — this is the mechanism that keeps the scale-drift class of defect from recurring.

2.4 Section weight and "critical" flags

When a script is built, each section can be given a weight and marked as critical. Both are saved with every call (`lib/services/scoring.ts:555–556`).

Weight now drives the overall score, as described in §2.1. With five sections weighted 20 / 5 / 25 / 25 / 25 and scored 90 / 40 / 80 / 80 / 80, the overall is 80; a plain average would give 74.

CONVENTION — NOT ENFORCED BY THE SYSTEM

Weights are expected to add up to **exactly 100%**. Only the create-script form checks this: on Rubric Config and in the Script Builder, the save button stays disabled until the total is 100 (`app/[locale]/(admin)/admin/settings/page.tsx:413–418`). Neither the API (`POST /api/scripts`) nor the database checks it, so a script created or edited any other way can carry any total.

The formula divides by the sum of the weights, so a total other than 100 still yields a valid score — the weights simply act as proportions. The rule exists so that the numbers on screen read as percentages.

KNOWN DEFECT — ALREADY REPORTED

The critical flag still does not affect the overall score. Marking a section critical does not penalise the call; the flag is only stored and used for display and alerts (§9). `criticalNames` (`lib/services/scoring.ts:466`) decorates the stored output and is never passed to `computeOverallScore()` .

`SCHEMA.md` describes the overall score as a "weighted average of section scores". That is now the actual behaviour.

2.5 Call outcome: Stage 1 and Stage 2

The product tracks two separate notions of "did this deal happen", and conflating them is a common source of confusion.

Stage 1 — Initial Result

How the call ended. As of migration `105_call_outcome_2_values.sql` there are only two values:

Value	Meaning
<code>closed</code>	The deal advanced on this call
<code>not_closed</code>	The call finished without the deal advancing

```
lib/types.ts:2 — export type CallResult = "closed" | "not_closed"
```

CHANGED SINCE v1.0 — READ THIS BEFORE ANSWERING ANY CLOSE-RATE QUESTION

The enum used to have four values. Migration 105 collapsed it to two and **rewrote historical data**:

Old value	New value
<code>closed</code>	<code>closed</code> (unchanged)
<code>partial</code>	<code>closed</code> — advanced partially counts as "closed something"
<code>not_closed</code>	<code>not_closed</code> (unchanged)
<code>no_outcome</code>	<code>not_closed</code> — an ambiguous result did not, in fact, close

Legacy string normalisation was remapped to match (`lib/constants.ts` — `LEGACY_OUTCOME_MAP`), so `follow_up` / `follow-up` now normalise to `closed` , and `no_decision` / `no_close` / `no-close` to `not_closed` .

Consequence: every close rate in the product is now structurally **higher** than the same period computed under the old scheme, because follow-ups count as closes. If a client says "our close rate jumped and we didn't change anything", this is the answer. It is a definitional change, not a performance change or a bug. The migration backfilled historical rows, so the product is internally consistent — but any number recorded outside the product before the migration will not reconcile.

Stage 2 — Actual Close

Whether the client actually paid: `paying` , `not_paying` , or `pending` (`lib/types.ts:119`). It is set in two ways:

- **Automatically from the CRM.** When a GoHighLevel opportunity becomes **Won** — through the webhook or the daily `sync-ghl-opportunities` cron — one call of that contact is marked `paying` : the most recent sales call, or the most recent call if there is none. `became_paying_at` is set to `ghl_won_at` : the Won date when GHL sends it (`lastStatusChangeAt`), otherwise the moment of the marking (`COALESCE(c.ghl_won_at, now())` , `scripts/117_stage2_from_ghl_won.sql:60`). This happens only if that call has no Stage 2 value yet and no call of the same contact is already `paying` . Each marking writes to `calls_data_corrections` with `applied_by = 'ghl_won_sync'` (`public.mark_stage2_paying_from_won` , migration `117_stage2_from_ghl_won.sql`).
- **Manually by the owner**, on the call detail screen (`app/api/calls/[id]/stage2/route.ts`).

The manual value always prevails. The automatic path never overwrites a call that already has a Stage 2 value, `pending` included.

OFTEN MISTAKEN FOR A BUG

198 paying calls have no `became_paying_at`, on purpose. The one-off backfill `118_stage2_won_backfill.sql` (25 September 2026) marked `paying` on one call for every contact already Won in GHL. It left `became_paying_at` **NULL** deliberately: until then `ghl_won_at` was rewritten on every sync, so it held the date of the last sync, not the date of the Won, and the real Won date was unknown. Only the automatic markings after that date record a date. In production on 30 September 2026 that is 9 calls since 26 September, all with a date. The backfill rows are in `calls_data_corrections` with `applied_by = '118_stage2_won_backfill'`.

WHY THIS EXISTS

A prospect saying "yes, let's do it" on the call and a prospect whose payment clears are two different business events, often weeks apart. Stage 1 is what the AI can detect from the conversation; Stage 2 is ground truth that only a human knows. Keeping them separate is what will eventually allow the system to learn which call behaviours predict **real revenue** rather than just verbal agreement.

OFTEN MISTAKEN FOR A BUG

Stage 2 is shown on the call detail screen and nowhere else. When the owner marks a call `paying` by hand, the route also stores an `intent_at_close` snapshot computed from the call's intent breakdown (`app/api/calls/[id]/stage2/route.ts:39-56`). A closed call is no longer forced to 5; that rule was removed. The automatic CRM path does **not** store `intent_at_close`. Nothing reads either value back: there is no report comparing predicted intent against who actually paid. If someone asks "where do I see my Stage 2 numbers?", the honest answer today is "nowhere yet". This is an incomplete feature, not a broken one.

2.6 Where calls come from

Two ingestion paths, both ending in the same place:

- **Manual upload** — the owner uploads an audio file or transcript through the upload screen.
- **CRM webhook** — GoHighLevel notifies the system when a call is recorded, and the pipeline fetches, transcribes, and analyses it automatically. This is the Pro plan feature.

Automatic ingestion can fail for reasons that have nothing to do with selling: no recording available yet, or a transcription failure.

A call that cannot be matched to a rep is no longer dropped. It is ingested, transcribed and scored like any other, and assigned to the organisation's system rep **Front Desk - AskMoses**

(`lib/constants/front-desk.ts` — `FRONT_DESK_NAME`, migration `109_front_desk_system_rep.sql`; `app/api/webhooks/ghl/route.ts:296-343`). The assignment is only logged (Vercel); it no longer sends a Slack alert. There are two cases, and only the first one is temporary:

Case	What happens
The payload has a <code>userId</code> , but it is not linked to any member	The call goes to Front Desk with its <code>ghl_user_id</code> stored. When that GHL user is linked to a member whose invite is accepted, their Front Desk calls move to the real rep automatically (<code>reassignFrontDeskCalls</code> , <code>lib/services/ghl-call-recovery.ts:42-55</code>). Migration is triggered by linking (<code>app/api/memberships/[userId]/route.ts:178</code>), by accepting the invite (<code>app/api/auth/verify-invite-token/route.ts:112</code>), and by a catch-up for calls that were still in the pipeline (<code>lib/services/chunk-pipeline.ts:572</code>).
The payload has no <code>userId</code> (typical of an inbound call from a new lead whose contact has no owner in GHL)	The call goes to Front Desk permanently . Migration selects calls by (Front Desk <code>trainer_id</code> , <code>ghl_user_id</code>) , and a call with no <code>ghl_user_id</code> never matches (<code>lib/db/calls.ts:910-918</code> — <code>dbGetFrontDeskCallsByGhlUser</code>).

In production, over the last 30 days to 30 September 2026, 123 of 271 inbound calls (45%) arrived with no `userId` , against 22 of 1,072 outbound calls (2%).

OFTEN MISTAKEN FOR A BUG

An inbound call is attributed to the contact's owner, not to whoever answered. The GHL workflow sends `userId` from the merge tag `{{phoneCall.user.id}}` (`docs/AskMoses-GHL-StepByStep.md`). For inbound calls, GHL does not record who picked up, so that user is the one **assigned to the contact** (the lead's owner), and the call goes to that rep. If the contact has no owner, `userId` is empty and the call goes to Front Desk permanently (table above). "The call went to the wrong rep" on an inbound call is usually contact ownership in GHL, not a defect in AskMoses. It also affects the transcript's speaker labels (§16 #15).

OFTEN MISTAKEN FOR A BUG

Calls that failed to process do not move the close rate or the score average. They never reach the classifier or the scorer: `is_sales_call` , `call_outcome` and `overall_score` stay `NULL` . They count in Total Calls, but are left out of the close-rate denominator (§3.1) and of every score average (§3.2). In production on 30 September 2026, all 95 `no_recording` calls and 31 of the 33 `transcription_failed` calls were in this state. A batch of failures shows up as volume without a result, so check `processing_status` when Total Calls grows and close rate and score stay flat.

2.7 Organisation scoping and roles

Every query is scoped to the active organisation. No screen ever mixes data across clients.

Role	Home after login	Can see	Cannot see
Rep (trainer)	My Page	Own calls and scores, own coaching	Team data, other reps, admin
Owner	Dashboard	Everything for their organisation	The SaaS admin panel, cost/COGS
Admin	Admin Panel	Everything, all organisations	—

Owners never see LLM cost or gross margin — those are filtered out of the billing responses **server-side**, by design, not by UI hiding.

3. The Owner Dashboard

The owner's home screen. Answers: is the team selling well, who needs attention, and what should we fix in the script?

3.1 Avg Close Rate **CALCULATED**

$\text{Avg Close Rate} = \text{closed sales calls} \div \text{sales calls with a result} \times 100$

Scope: the entire organisation, all time. Every counted call weighs exactly the same.

Denominator = `is_sales_call IS DISTINCT FROM false and call_outcome IS NOT NULL`. That keeps calls classified as sales and legacy calls never classified (`NULL`) that have a result. It leaves out calls classified as *Not a Sales Call* and calls with no result. Numerator: the same set with `call_outcome = 'closed'`. (`lib/db/calls.ts:778-811` — `dbGetOrgCloseRate`, both counts wrapped in `applySalesCallWithOutcome`, `lib/sales-calls.ts`.)

Every close rate in the product uses the same base: the card, the `trainers.close_rate` cache and its delta (`syncTrainerStats`), the Team Command Center, the trend lines, the rep's My Page, the Analytics outcome breakdown and leaderboard, and the AI insights. In-memory calculations use `closeRateOf / hasOutcome`. `toCall` still shows a call without a result as `not_closed` on screen, and marks it `hasOutcome: false`.

WHY THIS EXISTS

This used to be calculated as the average of each rep's individual close rate — which meant a rep with 1 call counted as much as a rep with 50. On real client data that distortion was large: one organisation showed 43% under the old method and 54% under the correct one; another showed 59% when the true figure was 43%. Counting call-by-call is the only way the number reflects the business rather than the shape of the team.

There is no date filter — it is deliberately a lifetime figure, so it is stable and hard to game.

OFTEN MISTAKEN FOR A BUG

Calls without a result are not in the denominator. A call that failed before it got a result (`no_recording`, `transcription_failed`, stuck in an intermediate status, or still `pending`) has `call_outcome = NULL`. It is not "not closed" — nobody knows — so it is left out. Before this change those calls counted as not closed, and in production (30 September 2026) they dragged the aggregate from 50.7% down to 45.1%, and one organisation from 52.9% down to 11.7%.

Pipeline failures therefore no longer move the close rate. They show up as calls without a result (`volume` and `processing_status`), which is where to look for them (§2.6).

OFTEN MISTAKEN FOR A BUG

The close-rate base and the call count are different. Total Calls (the sum of `trainers.total_calls`) counts every sales call, including those without a result. `closedCalls ÷ totalCalls` from Total Calls will not match the card.

The same applies to `call_stats_weekly` (`stamp_call_stats_weekly`, migration 107): its `total_calls` includes calls without a result, so `closed_calls ÷ total_calls` there is **not** the product's close rate. No screen reads that table today.

OFTEN MISTAKEN FOR A BUG

The delta underneath the card ("+7 pts since week 1") is **not measured over the same period** as the big number. The headline is all-time; the delta compares the first and last week within a window of at most six weeks and at most 200 calls. They are different questions — "how do we do overall?" and "are we trending up recently?" — shown next to each other without a visual cue.

3.2 Team Avg Call Score CALCULATED

Team Avg Call Score = average of each rep's average score (only reps with at least one **scored** call)

Each rep's average (`trainers.score`, `syncTrainerStats`) covers only calls with a valid score. A call is left out if its `overall_score` is `NULL`, or if its `scoring_status` is `scoring_failed` or `transcript_leaked` (`lib/db/trainers.ts:90` — `hasScoreRow`, plus `excludeFailedScoring` on the query).

A rep with no scored call is stored with score 0. The card leaves out reps with `score = 0` (`app/[locale]/dashboard/page.tsx:74`). That is equivalent to "no scored call", because a real average of 0 cannot happen: failed scoring is already excluded, and an evaluated call scores above 0.

WHY THIS EXISTS

Reps with zero calls have a stored score of 0. Including them would drag the team average down every time someone new is invited, making the metric jump for reasons that have nothing to do with performance. The same goes for calls without a score: counted as 0, they dragged the production average of sales calls from 60.1 down to 53.4 (30 September 2026), and one organisation's card from 2.0 down to 0.5.

Every score average in the product uses the same base: the rep's score and delta, the Team Command Center, the trend lines, the rep's My Page, and Analytics (trend, Master Coach, Rising Star, overall average). In-memory calculations use `avgScore0f / hasScore`. Section averages (`avgRubricScores`, Analytics' weakest sections, the revenue-leak insight) count only calls that have sections and valid scoring (`hasRubric`). Before this change, a call without sections counted as 0 in every section.

OFTEN MISTAKEN FOR A BUG

This one is an **average of averages**, unlike the close rate — so a rep with 1 call influences it as much as a rep with 50. That is inconsistent with how the close rate now works, and is a fair candidate for the same treatment.

3.3 Total Calls and Active Sales People **CALCULATED**

Total Calls is the sum of every rep's lifetime call count. Active Sales People counts reps whose invitation has been **accepted** — including those who have not yet made a call. These two use different populations, which is why the numbers can look mismatched (five active people, zero calls).

3.4 Correlation Engine — **MISLEADING LABEL**

Presented as "Correlation Engine — What Drives Closes", with High / Medium / Low correlation badges per rubric dimension.

What it actually shows: `score` = the team's average score in that dimension `badge` = that same score, banded (≥ 85 High, ≥ 70 Med, else Low)

```
lib/services/rubric.ts:327 — buildCoachingDrivers() maps each section to  
toCorrelationLevel(s.teamAvg) .
```

KNOWN DEFECT — **ALREADY REPORTED**

No correlation is calculated anywhere — not at any call volume. The badges are the team's average score wearing a statistical costume. The panel never compares closed calls against lost calls, which is what a correlation would require. "Objection Handling — High correlation" means only "the team scores above 85 in Objection Handling", not "Objection Handling drives closes".

Still present as of dev @ aafc10 .

This matters because it can **invert the advice**. If the team scores 60 in Objection Handling but nearly every won deal had a high score there, that is the single biggest lever in the business — and the panel would show it in red as "Low correlation", saying the opposite of the truth.

Below three calls (`MIN_CALLS_FOR_STATS = 3` , `components/shared/CorrelationEngine.tsx:17`) the panel honestly renames itself to team-averages language. Above three calls the statistical title takes over. Three calls is far too low a bar for that promise.

WHY THIS EXISTS

A real correlation is feasible — the data needed already exists on every call (per-dimension scores plus the outcome). The measure would be the difference in average score between calls that closed and calls that did not, per dimension. It has not been built yet; the current panel was the interim step.

3.5 Team Health CALCULATED

Status	Rule
Active	Last call within 1 day
Recent	Last call within 7 days
Away	No call in over 7 days, or never

The close rate and delta shown here are **not recalculated live** — they are read from a cached row on the `trainers` table, written by `syncTrainerStats()` (`lib/db/trainers.ts`) and refreshed only when a new call is analysed.

OFTEN MISTAKEN FOR A BUG

If a rep's stats look stale, this cache is why. It refreshes on call analysis, not on page load. A call that entered through an unusual path (a seed, a manual retry) may not have triggered a refresh. A resync is available at `/api/sync-trainers`, and it correctly zeroes a rep who has no qualifying calls rather than leaving stale numbers behind.

OFTEN MISTAKEN FOR A BUG

A delta of exactly zero shows an **upward green arrow**, because the rule is "zero or above = up". A rep with no change looks like a rep improving.

3.6 Close Rate Trend CALCULATED

Up to six weeks of team close rate, one point per week, weeks running Monday to Monday.

WHY THIS EXISTS

A week with no calls shows as a **gap** in the line, not a zero. Plotting it as 0% would read as "we closed nothing that week", when the truth is "nobody called that week". The distinction changes what an owner does next, so the chart refuses to guess.

For a rep whose calls all fall within a single week, the chart switches to a per-call cumulative view (C1, C2, C3...) instead of weekly points, because a single weekly point cannot draw a line.

3.7 Score by Sales Person CALCULATED

A grid of reps against rubric dimensions. The highest score in each column is highlighted green; anything below 70 shows red. Based on the 200 most recent calls.

NOTE ON THE UNDERLYING AVERAGES

The per-section averages come from the `trainers` cache, and the aggregation at `lib/db/trainers.ts:89` still applies `raw > 5 ? raw : raw * 20` — the legacy scale-normalisation rule. See §16, "A note on scale drift".

3.8 AI Insights **AI OPINION** **CALCULATED**

Four cards mixing real arithmetic with AI-written prose. The numbers inside them are computed in code; the sentences around them are generated. The "Coaching ROI" card quotes the close rate from the same source as the headline card, so those two always agree.

KNOWN DEFECT — ALREADY REPORTED

The **"Revenue Leak" card is broken by a scale error**. It compares section scores — stored 0–100 — against a threshold of 3.5, written for the 0–5 scale.

```
lib/services/insights.ts:88 — if (avg < 3.5) trainersBelow++
lib/services/insights.ts:96–97 — c.rubricScores[weakest.key] < 3.5 / >= 3.5
lib/services/insights.ts:122 — the summary string "${trainersBelow} of ${sorted.length}
sales people score below 3.5 on ${weakest.label}"
```

Only a tiny fraction of section scores fall below 3.5 on a 0–100 scale, and those are catastrophic (2/100), not mediocre. The result is that the card almost always reads "0 of N sales people score below 3.5", and prints sentences like "Marcus scores 92/5". **Still present as of dev @ aafcf10** — the file was touched by the outcome migration, but the thresholds were not corrected.

4. Analytics

A deeper cut of call data. Everything on this page is calculated in the browser from real calls — there is no AI involvement at all.

4.1 Performance Trend CALCULATED — Average call score grouped by day.

4.2 Top Improvement Areas CALCULATED — The average of each rubric section across all loaded calls, sorted worst-first.

4.3 Outcome Metrics CALCULATED — Counts of `closed` and `not_closed`, plus a close rate using the same rule as the dashboard. *(The `partial` and `no_outcome` buckets were removed here alongside migration 105 — see the diff to `lib/services/insights.ts`.)*

4.4 Trainer Conversion Leaderboard CALCULATED — Close rate per rep, ranked. Bars turn green at 50% and above.

OFTEN MISTAKEN FOR A BUG

This leaderboard groups reps by **display name**, not by their unique ID. Two reps with the same name would be merged into one row. Unlikely, but worth knowing.

4.5 Achievements CALCULATED

Badge	Rule
Master Coach	Rep with the highest average score
Perfect Calls	First rep with a call scoring 95 or above
Rising Star	Compares the last 3 calls against the first 3 calls

OFTEN MISTAKEN FOR A BUG

"Rising Star" compares the last three calls **of the whole organisation** against the first three of the whole organisation, then credits the rep who made the most recent call. It can therefore name a rep based on other people's improvement.

`app/[locale]/dashboard/analytics/page.tsx:168-180` — `sorted.slice(-3)` vs `sorted.slice(0, 3)`, crediting `recent[recent.length - 1].trainerName`. It is a demo-grade heuristic, not a real measure.

OFTEN MISTAKEN FOR A BUG

This page does not paginate. It loads roughly the **1,000 most recent calls** and analyses those. An organisation past that volume will see analytics based on a truncated window, with no warning on screen.

5. Script Intelligence (Insights)

Reads recent transcripts and proposes an improved version of the script. The owner approves or rejects, and approving activates the new script in production.

Sample: up to **7** recent calls with transcripts, each truncated to 1,500 characters.

KNOWN DEFECT — ALREADY REPORTED

Almost every number on this screen is the language model's opinion, presented with the visual authority of a measurement:

- **Playbook Health Score (0–100)** is asked for directly in the prompt — `lib/script-intelligence/analyze.ts:25` — and passed straight through at line 230. No code computes or validates it.
- **Per-section scores** are likewise the AI's read.
- **The effectiveness label** (`good` / `roomToImprove` / `poor`) is *instructed* to follow the health score — `analyze.ts:55` : "good if healthScore >= 80, roomToImprove if >= 60, poor otherwise" — but the code never checks that it does. Line 231 passes `parsed.effectivenessLabel` through unverified, so a model that mislabels itself puts the wrong label on screen.
- **Most seriously, the uplift figures on top closer phrases (" +18%") are invented.** `analyze.ts:47` asks for a string "uplift": "string — e.g. +18%" with no instruction on how to derive it, and no uplift is calculated anywhere in the system.

Still present as of dev @ aafcf10 .

This matters more than elsewhere because these numbers sit next to buttons that really do swap the organisation's active script. A business decision is being made on the strength of figures that have no statistical basis.

WHY THIS EXISTS

The feature is genuinely useful as **qualitative** analysis — the AI reading seven real transcripts and suggesting sharper language is real value. The problem is presentation: qualitative judgement dressed as quantitative evidence. Reframing these as "AI assessment" rather than scores and percentages would keep the value and drop the false precision.

6. Script Gap Detection

Finds friction points in real conversations that the current script does not address, and proposes targeted rewrites.

Sample: exactly **3** calls with full transcripts. Results are cached for 7 days, then recomputed on next view.

Frequency badge CALCULATED

```
frequency = calls where the gap appears ÷ calls analysed × 100
```

WHY THIS EXISTS

This is the one AI feature in the product where the percentage is **honest**. The model identifies which calls show the friction, but the percentage is then computed in code from that count — it is not a number the AI made up. Gaps the model reports without pointing to a real call are discarded before saving. **This is the pattern the other AI features should follow.**

OFTEN MISTAKEN FOR A BUG

With a denominator of 3, the only possible values are 33%, 67%, and 100%. The badge is colour-coded as if it were a fine-grained statistic (red at 60% and above), but it is really reporting "this appeared in 1, 2, or 3 conversations". The percentage is accurate; the precision it implies is not.

The severity rating (high / medium / low) is the AI's judgement, validated only against the list of allowed values.

7. History

A searchable table of analysed calls. No aggregate metrics — it displays stored scores and outcomes without recomputing anything.

Calls from the same contact are grouped into a single row with a "View All" action. Paginated at 10 per page in the browser.

8. The Rep's Dashboard (My Page)

The individual seller's view. Deliberately excludes coaching notes written for the owner's eyes.

8.1 The four KPI cards **CALCULATED**

Score, Close Rate, Calls, and Closed — each with a window selector of 2, 4, or 6 weeks, defaulting to 6.

Within the selected window: $\text{score} = \text{weighted by the number of scored calls each week, not a plain average of weeks}$ $\text{close rate} = \text{total wins} \div \text{total calls}$ **with a result** in the window

The Calls card still shows every call in the window. Calls without a score or a result are left out only of the score and close-rate figures (§3.1, §3.2).

WHY THIS EXISTS

Weighting by volume means a week with 12 calls influences the number more than a week with 1. Averaging the weekly percentages instead would let a single-call week swing the figure wildly — one lucky call would read as a 100% week and drag the average up.

OFTEN MISTAKEN FOR A BUG

The delta compares the selected window against **one single week** immediately before it — not against an equivalent window. So "last 6 weeks" is compared to the 7th week alone. That is a noisy comparison and arguably the wrong one, but it is the current design.

OFTEN MISTAKEN FOR A BUG

A delta of exactly zero is **hidden** rather than shown as "0". Reps sometimes read the missing delta as a bug.

8.2 Empty weeks

A week with no calls is flagged as empty rather than scored as zero, and is excluded from comparisons — the same reasoning as the trend chart in §3.6.

9. Calls and Call Detail

The team's call list and the full analysis of a single call.

The detail screen shows the overall score, per-section bars with the AI's written justification, strengths, improvements, the intent breakdown, the outcome badge, and — for owners only — coaching notes and the Stage 2 marker.

OFTEN MISTAKEN FOR A BUG

The Calls list opens filtered to **Closed** and **Not Closed** (`app/[locale]/calls/CallsTable.tsx:50` — `DEFAULT_RESULT_FILTER`). Calls classified as *Not a Sales Call* are hidden until they are ticked in the result filter. Calls still being analysed are always shown, whatever the filter says. "A call is missing from the list" is usually the default filter.

OFTEN MISTAKEN FOR A BUG

Calls that are not scored, by design. Before scoring, the AI classifies each call as a sales call or not (the *SALES CALL GATE* in `lib/services/scoring.ts:135-140` , same criterion in `lib/services/sales-call-classifier.ts`). The prompt treats any recording "where no selling activity is taking place" as not a sales call, and says to prefer *true* when in doubt. In practice this catches voicemails, messages left for someone, and logistics calls or calls with existing customers.

Such calls are saved with `is_sales_call = false` , no score, outcome or sections (`lib/services/ghl-call-scoring.ts:212-221`), and show as *Not a Sales Call*, hidden by the default filter. They are excluded from the close rate (`dbGetOrgCloseRate` , `lib/db/calls.ts:768` — `applySalesCallOnly`) and from every other aggregate that uses `applySalesCallOnly` . This is intentional: counted as `not_closed` , they would drag the close rate down for reasons that have nothing to do with selling.

This is the model's judgement, not a hard rule. Since 20 September 2026, 204 of the 210 transcripts that look like a voicemail were classified as not a sales call. The other 6 were scored.

Triage: filter Calls by the rep and tick *Not a Sales Call*. The list is newest first, and calls with the same contact are grouped behind **View All**. Open the call: the transcript is shown even when there is no score. Escalate only if the call is not in the list at all.

OFTEN MISTAKEN FOR A BUG

A section marked **critical** raises a red alert when it scores **40 or below** on this screen (`components/shared/CallDetail.tsx:186` — `section.critical && section.score <= 40`). The coaching email uses a different threshold — **60** (`lib/email/coaching-template.ts:115,123`) — so the email flags calls the screen does not. Same concept, two numbers, because they were built separately.

KNOWN DEFECT — ALREADY REPORTED

The **"Coaching notes"** textarea is not wired to anything.

components/shared/CallDetail.tsx:325–337 renders a `<textarea>` with no `value` , no `onChange` , no save action and no persistence — only focus/blur border styling. Anything typed into it is lost on navigation. It is leftover demo UI. **Still present as of dev @ aafcf10** .

10. Team Command Center

The owner's per-rep deep dive: behavioural profile, best and worst calls, and AI coaching recommendations.

10.1 Behavioural profile CALCULATED

For each dimension of the active script: the rep's average, the team's average, and the gap between them.

WHY THIS EXISTS

The dimensions come from the organisation's **active script** rather than from each rep's own calls, so that every rep is measured on the same axes and can be compared horizontally. If each rep's chart used their own call history, the charts would not line up.

10.2 Cached stats and deltas

OFTEN MISTAKEN FOR A BUG

The rep close rate, score, and both deltas rendered in `components/shared/TrainerTabs.tsx:151–174` all come from the `trainers` cache described in §3.5, not from a live recomputation. A rep whose latest calls have not triggered a `syncTrainerStats()` run will show outdated figures across the whole card. Where a screen recomputes the headline live but leaves the delta on the cache, a correct score can sit next to a stale delta.

10.3 Coaching recommendations AI OPINION

The metrics fed into the AI prompt are computed in code from the rep's last **20** calls. The recommendations that come back are free text, and the prompt forbids citing specific calls. No fabricated statistics appear in the output.

OFTEN MISTAKEN FOR A BUG

The close rate used inside these recommendations covers the rep's **last 20 calls**, while the dashboard shows their **lifetime** rate. The same rep can therefore be described as closing at 45% in a recommendation while the dashboard says 38%. Both are correct for their scope.

OFTEN MISTAKEN FOR A BUG

If the AI call fails, the system silently falls back to pre-written recommendations belonging to one of **four fictional demo personas** (`lib/services/coaching.ts:20–27` — `marcus`, `jamie`, `jordan`, `taylor`, selected by hashing the trainer ID). A real rep can be shown generic advice with no indication that it is not personalised.

11. Intent Analysis

Scores how ready each prospect was to buy, independent of how well the rep performed.

11.1 The Intent Index **CALCULATED**

The AI reads the transcript and rates four signals from 0 to 10: **Financial**, **Urgency**, **Authority**, and **Engagement** (`lib/services/scoring.ts:440-454` , clamped to 0–10 integers at lines 272–278).

```
Intent Index = (  $\sum$  signal  $\times$  weight  $\div$   $\sum$  weights )  $\div$  2
```

Result: 0 to 5, one decimal. `lib/utils/intentScore.ts` — `computeIntentIndex()`

The formula normalises by total weight, so it is invariant to the weight base — legacy base-10 weights and current base-100 weights produce the same result.

WHY THIS EXISTS

Separating intent from score answers the question every sales manager asks: **was this a bad call or a bad lead?** A rep with a low close rate and consistently low intent has a marketing problem, not a coaching problem. This is also what makes the Marketing Intelligence feature possible.

The four weights default to 25% each (`lib/constants/intent.ts` — `DEFAULT_INTENT_WEIGHTS`) and can be configured per organisation. The weights used are snapshotted onto each call at analysis time, so changing them later does not silently rewrite history.

OFTEN MISTAKEN FOR A BUG

The four signals are described **two different ways** in the product. The prompt that instructs the AI defines Financial as *"Does the prospect have budget available or mention budget concerns?"* (`lib/services/scoring.ts:402`), while the on-screen help text for the same signal asks *"Did the person react to the price with curiosity or demonstrate resistance?"* (`messages/en.json` — `signals.financial.question`). Budget availability and reaction to price overlap but are not the same thing. All four signals have this gap between the internal scoring definition and the customer-facing description.

The customer guide now uses wording that covers both readings, but the two should still be reconciled so the AI scores exactly what the interface claims it scores.

11.2 When intent cannot be measured

FIXED SINCE v1.1

The rule "**closed** ⇒ **intent 5**" **no longer exists**. It lived in three places, and all three were removed:

- the scoring prompt (`If the deal closed, intent is 5.` , formerly in `app/api/analyze/route.ts`);
- the Stage 2 route, which forced `intent_at_close = 5` for closed calls;
- the constant `INTENT_RULES.CLOSED_CALL_INTENT` in `lib/constants/intent.ts` .

Intent for a closed call is now read from the conversation like any other. The helper `deriveCallIntentBreakdown()` (`lib/utis/intentScore.ts`), which turned a closed call into a flat 10/10/10/10, is still in the file but has no callers.

Calls analysed before the change may still carry intent 5 from the old rule (migration 087 also set `intent = 5` on every closed call) unless they were recalculated since.

OFTEN MISTAKEN FOR A BUG

For calls analysed before this feature existed, the breakdown shown is a flat 5/5/5/5 placeholder. Those calls are excluded from the "Highest Priority Leads" table but are included in the radar chart averages, diluting them toward the middle with no visual marker.

11.3 The "Won" column

OFTEN MISTAKEN FOR A BUG

The Won column on the priority leads table reads the **CRM's opportunity status** (`ghlWonStatus`), not the Stage 2 `paying` marker. Since migration 117 a CRM Won also marks one call of the contact as `paying` (§2.5), so the two usually agree. They can still differ: a call marked `paying` by hand without a Won in the CRM does not show as won here. And when a contact has several calls, only one of them becomes `paying` .

12. Appointments and Call Date

New since v1.0 — not covered by the previous edition of this manual.

12.1 Call Date

`calls.call_date` (migration 036) is the date the conversation **actually happened**, distinct from `date_created_at` (= upload/ingestion time). The Intent Analysis dashboard column that displayed this was renamed from **"Eval Date" to "Call Date"** (commit `f11bfcd`).

The field carries a provenance marker: `'ghl'` when the call arrived by webhook (reliable), `'llm'` when it came from manual upload and the date was estimated from the transcript. The UI flags the `llm` case as a fallback.

12.2 Appointment sync `CALCULATED`

Leads' scheduled appointments are pulled from GoHighLevel by a scheduled job (`app/api/cron/sync-ghl-appointments/route.ts` , registered in `vercel.json`) into an `appointments` table (migration 094), and joined to calls by `contactId` .

Two fields surface on the call (`lib/types.ts`):

Field	Meaning
<code>appointmentAt</code>	When the lead's appointment is booked for
<code>appointmentStatus</code>	<code>booked</code> / <code>confirmed</code> / <code>cancelled</code> / <code>showed</code> / <code>noshow</code>

OFTEN MISTAKEN FOR A BUG

This enrichment is **opt-in per request**. It is only populated when the consumer explicitly asks — `GET /api/calls?withAppointments=true` . The two states are distinct and deliberately so:

- `undefined` = **not queried** (the caller did not ask for appointments)
- `null` = **queried, and there is no appointment**

A screen showing no appointment data is not necessarily a sync failure — check whether that screen requests the enrichment at all.

OFTEN MISTAKEN FOR A BUG

The sync is a **scheduled job, not a webhook**. An appointment booked or cancelled in the CRM moments ago will not appear until the next run. "The appointment status is wrong" is usually "the appointment status is stale".

13. Marketing Intelligence

Reads the best closed calls and suggests ad copy — headlines, angles, and objections to pre-empt.

Sample: **3 to 5** calls, chosen at random from the 10 highest-scoring closed calls. Refreshes weekly (cached 7 days).

WHY THIS EXISTS

Sampling only won calls, and only high-scoring ones, is deliberate: the goal is to extract the language that works when everything goes right, then feed it back into acquisition. It is **not** meant to be representative of all calls.

Confidence percentage AI OPINION

KNOWN DEFECT — ALREADY REPORTED

The confidence figure (0–100) shown on each suggestion is **the AI grading its own answer**. The prompt literally asks for *"confidence (integer 0–100, your honest read of how strong the signal from the calls is)"* — `lib/services/marketing-intelligence.ts:79`. Code only checks it is a finite number (line 146) and bands it into High/Medium/Low at 80/60 (`levelFor()` , lines 121-123); nothing verifies it against sales data. It is then rendered as a progress bar, which reads as a statistical confidence interval. It is not one. **Still present as of dev @ aafcf10**.

The "based on N calls" badge **is** real — it reflects the actual sample size.

14. Admin Panel and Billing

14.1 What the admin panel shows

Four cards: Total Organisations, Pending Approvals, Total Calls, and Average Score.

KNOWN DEFECT — ALREADY REPORTED

The **Average Score card renders as a percentage** — for example "4.2%".

`app/[locale]/(admin)/admin/page.tsx:68` — `value={ ${toDisplay5(metrics.avgScore)}% }` . The value is converted to the 0–5 scale and then given a "%" suffix, which is neither the 84% it would be on the stored scale nor the "4.2/5" that was intended. Cosmetic, but visible on the first screen an admin sees. **Still present as of dev @ aafc10** .

14.2 The organisations grid — Minutes, not Cost

The SaaS Panel grid shows each organisation's **billable minutes consumed this month**

(`Client.billableMinutesThisMonth`), computed with the same rule the Billing feature charges by — `ceil` per call, calls under 30s excluded.

FIXED SINCE v1.0

The previous edition documented a high-severity defect here: the grid calculated cost at **\$2.00/minute** while the billing screens used \$0.0667, so the same organisation showed costs ~30× apart depending on which admin screen you opened.

This is resolved. `COST_PER_MINUTE_USD = 2` and the `secondsToCostValue()` / `formatCost()` helpers were **deleted** from `lib/billing.ts` ; `Client` dropped `totalCostThisMonth` in favour of `billableMinutesThisMonth` ; and the `COST` column was removed from the SaaS Panel (commit `a4df35a`). **Money now lives in exactly one place: `/admin/billing` .**

One caveat worth knowing: `lib/billing.ts` now duplicates the minutes rule from `lib/db/billing.ts` (the authority) on purpose, and says so in its header. `tests/billing-format.test.ts` locks the key cases (30s floor, ceil per call) so drift between the two shows up as a test failure.

14.3 MRR **PLACEHOLDER**

OFTEN MISTAKEN FOR A BUG

MRR is typed in by hand when an organisation is created, and can be overridden manually afterwards. It is not connected to Stripe, and it is **never summed anywhere** — no dashboard adds it up. The billing model moved to per-minute consumption and MRR was left behind as a loose field. If someone asks "where do I see total MRR?", the answer is that no such screen exists.

14.4 Billing status

`organizations.billing_status` drives the badge on the Billing screen and is independent of `subscription_status`.

Values: `PAID` (charged), `PILOT`, `DEMO`, `DISABLED`.

CHANGED SINCE v1.0 — migration `106_org_billing_status_default_paid.sql`

The column was created (migration 082) with `DEFAULT 'PILOT'`, on the assumption that new orgs started as free pilots and someone would promote them later. **No write path ever existed** — no route, screen, or trigger changed the value — so no organisation ever left `PILOT`, and the admin saw zero revenue against orgs that were consuming minutes and generating LLM cost.

The default is now `PAID`; `PILOT` / `DEMO` / `DISABLED` are the explicit exception, set by an admin in `/admin/billing` (the `BillingTable` dialog writes via `PATCH /api/admin/organizations/[id]/billing-rate`).

The migration deliberately does not backfill. Organisations already on `PILOT` stay on `PILOT` — flipping them en masse would begin billing existing clients retroactively, which is a commercial decision, not a migration. Admins promote them one at a time. If a client asks why a long-standing org still shows as `PILOT`, this is why.

14.5 Billing — how clients are charged `CALCULATED`

`billable minutes` = call duration rounded **UP** to the next whole minute calls under **30 seconds** are not billed at all `cost` = `billable minutes` × the organisation's rate

Default rate: **\$0.0667 per minute** ($\approx$ \$1 per 15-minute call) `lib/billing.ts` —
`MIN_BILLABLE_SECONDS = 30`, `billableMinutes()` = `Math.ceil(durationSeconds / 60)`
`lib/db/billing.ts:37` — the default rate description

WHY THIS EXISTS

Rounding up per call rather than summing raw seconds is standard telecom-style billing and is simple to explain on an invoice. The 30-second floor exists so that misdials and instant hang-ups — which cost the platform almost nothing — do not appear as charges and trigger support tickets.

The rate is configurable per organisation, which supports negotiated deals.

NOTE

`MIN_BILLABLE_SECONDS` (billing floor) and `MIN_ANALYZABLE_CALL_SECONDS` (ingest floor, `lib/constants/limits.ts`) both happen to be 30 today. That is coincidence, not coupling — changing one should not change the other.

14.6 LLM cost CALCULATED

$$\text{cost} = (\text{input tokens} \times \text{input price} + \text{output tokens} \times \text{output price}) \div 1,000,000$$

Prices live in a versioned database table. Editing a price never overwrites the old one — it deactivates it and inserts a new row with an effective date, so historical costs stay accurate. Current active pricing covers OpenAI (GPT-4o, GPT-4o-mini, GPT-4-turbo, GPT-4, GPT-3.5-turbo, Whisper) and Gemini (2.5 Pro, 2.5 Flash, 2.5 Flash-Lite, 2.0 Flash, 2.0 Flash-Lite).

WHY THIS EXISTS

This replaced an earlier estimate that simply assumed cost was 30% of revenue. Margin figures in the admin panel are now real, measured per API call.

Owners never see this. It is admin-only, filtered server-side.

14.7 Plan limits CALCULATED

Plan	Sales people	Calls per month	Key features
Starter	5	200	Manual upload, AI analysis, coaching email
Pro	15	1,000	Adds CRM webhook, automatic ingestion
Pro + RAG	Unlimited	Unlimited	Adds knowledge base, vector search

scripts/029_plan_limits.sql:27-29 — pro_rag stores NULL for both limits, meaning unlimited.

WHY THIS EXISTS

These limits are enforced by the **database itself**, not by the application, using locks that make the check atomic. That means two simultaneous invitations cannot both slip past the seat limit — a race condition that application-level checks would not catch. **Owners do not count against the seat limit; only reps do.**

The monthly call limit counts from the first of the calendar month, in UTC. There is no closing job — the month simply rolls over.

15. Fixed Since v1.0

If you are working from a previous PDF, unlearn these entries. They are no longer defects.

# in earlier edition	What it was	Status now
1 (v1.1)	Section weights saved but never used; overall score was a plain average	Fixed for weights. Overall score is the weighted average, with a plain-average fallback (<code>lib/services/overall-score.ts</code> , §2.1). The critical flag is still unused — it stays in §16 as #1.
(§11.2, v1.1)	Closed calls forced to intent 5 (prompt, Stage 2 route, <code>INTENT_RULES</code>)	Removed. See §11.2.
13 (v1.1)	Stage 2 set only by hand	Partly changed. A CRM Won now marks a call <code>paying</code> automatically (§2.5). The missing report is still open — §16 #13.
4	Admin organisations grid calculated cost at \$2.00/min against the real \$0.0667/min — two admin screens disagreeing by ~30x	Fixed. <code>COST_PER_MINUTE_USD</code> deleted from <code>lib/billing.ts</code> ; <code>COST</code> column removed from the SaaS Panel (<code>a4df35a</code>); the grid now shows billable minutes and money lives only in <code>/admin/billing</code> . See §14.2.
5	Intent Analysis dashboard ignored configured weights, always using 25/25/25/25, because it fetched from an endpoint that did not exist in production	Fixed. <code>components/shared/IntentDashboard.tsx:84</code> now resolves weights in-process via <code>resolveIntentWeights(signals)</code> (<code>lib/utils/intentScore.ts:33</code>) with a documented default, and <code>app/api/stage-config/route.ts</code> is a real, auth-scoped route.
(scale drift note)	The <code>value <= 5</code> → multiply by <code>20</code> rule was described as present "in several places"	Largely cleaned. <code>lib/score-display.ts</code> was introduced as the single source of truth for scale conversion, and its header forbids inline <code>/ 20</code> or <code>*</code> <code>20</code> elsewhere. Two sites still apply the legacy rule — see §16.

16. Known Defects — Do Not Report These

Every item below was re-verified against `dev @ aafc10` at the `file:line` given; #1 and #13 again at `18807ca`, #15 at `66cf2c6`. They are already identified. Reporting them again costs everyone time — but if you see behaviour that is not on this list and not explained elsewhere in this manual, that is worth raising.

#	Where	What is wrong	Evidence	Impact
1	Scoring engine	The "critical" flag is saved but never affects the overall score (weights now do — see §15)	<code>lib/services/scoring.ts:466,538</code>	Medium — a configuration option that silently does nothing
2	Dashboard → Correlation Engine	Calculates no correlation; shows average score with correlation labels	<code>lib/services/rubric.ts:327</code>	High — can point coaching in the wrong direction
3	Dashboard → Revenue Leak insight	Compares 0–100 scores against a 3.5 threshold meant for the 0–5 scale; prints "92/5"	<code>lib/services/insights.ts:88,96,97,122</code>	High — card is statistically meaningless
4	Script Intelligence	Health score, section scores and "+18%" uplifts are AI opinion presented as measurement; effectiveness label never validated	<code>lib/script-intelligence/analyze.ts:25,47,55,231</code>	High — drives a real script-activation decision
5	Admin panel	Customer Health is always "healthy"; no rule ever changes it	Written as <code>'healthy'</code> at <code>app/api/onboarding/organization/route.ts:176</code> and <code>app/api/organizations/route.ts:230</code> ; no <code>UPDATE</code> path exists	Medium — decorative field
6	Call detail	Coaching notes textarea does not save — no <code>value</code> , no <code>onChange</code> , no persistence	<code>components/shared/CallDetail.tsx:325–337</code>	Medium — looks functional, silently loses input

#	Where	What is wrong	Evidence	Impact
7	Team Command Center	Rep stats and deltas read from a cache refreshed only on call analysis; a correct score can sit beside a stale delta	<code>components/shared/TrainerTabs.tsx:151–174</code> , <code>lib/db/trainers.ts</code>	Medium — stale numbers with no indication
8	Coaching recommendations	On AI failure, silently shows advice from one of four fictional demo personas	<code>lib/services/coaching.ts:20–27</code>	Medium — no indication it is not personalised
9	Marketing Intelligence	Confidence % is the AI grading itself, rendered as a statistical bar	<code>lib/services/marketing-intelligence.ts:79</code> , <code>121–123, 146</code>	Medium — false precision
10	Admin panel	Average Score renders as "4.2%" — a 0–5 value with a percent sign	<code>app/[locale]/(admin)/admin/page.tsx:68</code>	Low — cosmetic
11	Critical section alerts	Screen alerts below 40, coaching email alerts below 60	<code>components/shared/CallDetail.tsx:186</code> vs <code>lib/email/coaching-template.ts:115, 123</code>	Low — email over-alerts relative to the app
12	Analytics → Rising Star	Compares organisation-wide first/last calls, credits the most recent rep	<code>app/[locale]/dashboard/analytics/page.tsx:168–180</code>	Low — demo-grade heuristic
13	Stage 2 (Actual Close)	Filled automatically as <code>paying</code> on a CRM Won (manual value prevails), but no report consumes Stage 2 or <code>intent_at_close</code> — only the call detail marker shows it	<code>scripts/117_stage2_from_ghl_won.sql</code> , <code>app/api/calls/[id]/stage2/route.ts:39–56</code>	Low — incomplete, not broken

#	Where	What is wrong	Evidence	Impact
14	Buying Intent	Each signal is defined one way in the AI prompt and described another way in the on-screen help	<code>lib/services/scoring.ts:402</code> vs <code>messages/en.json</code> → <code>signals.financial.question</code>	Low — the AI may not score exactly what the UI claims
15	Transcripts	Speaker labels (Trainer / Prospect) are sometimes swapped. Reported more often on inbound calls.	<code>lib/services/whisper.ts:392–414</code> — <code>assignSpeakerLabels()</code> ; name hint from <code>lib/services/chunk-pipeline.ts:451</code>	Medium — scoring reads the labelled transcript, so the AI can credit or blame the rep for what the customer said

Renumbered from v1.0: former #4 (billing rate) and #5 (intent weights) are fixed and moved to §15; the remaining items shifted up. v1.2 keeps the numbering: #1 and #13 were reworded, not removed. v1.3 adds #15.

A note on speaker labels (#15)

Whisper does not separate speakers. The labels come from a second, **text-only** pass: an LLM (the active model, default `gpt-4o-mini`) reads the plain transcript and decides who said each line (`lib/services/whisper.ts:381–414`). It has no acoustic cues. It infers roles from content, from two optional name hints ("The salesperson's name is ...", "The prospect's name is ..."), and from a default: a line it cannot place is labelled `Trainer:`, because "sales call recordings usually start with the salesperson".

Inbound calls are where each of those inputs is weakest:

- the salesperson name hint is the call's `trainer_name`, which on an inbound call is the contact's owner and not necessarily who answered (§2.6), or `Front Desk – AskMoses` when the call went to Front Desk;
- an inbound call opens with the customer explaining why they are calling, which reads like the salesperson's discovery.

The frequency on inbound is reported from use, not measured. The mechanism above is what the code shows.

A note on scale drift

Several defects in the product's history share one root cause. The product originally scored calls on 0–5 and later migrated to 0–100. During that migration, a temporary rule was added in several places: "if the value is 5 or less, assume it is on the old scale and multiply by 20". The migration finished; the rule was never fully removed.

Where it still lives, as of `dev @ aafcf10`:

- `lib/db/trainers.ts:89` — `sectionSums[col].sum += raw > 5 ? raw : raw * 20`, in the rubric section aggregation that feeds the `trainers` cache
- `lib/services/coaching.ts:42` — `return v > 5 ? v : v * 20`

The rule **inverts precisely the worst calls**: a genuinely catastrophic section score of 3/100 becomes a comfortable 60/100, which is exactly the call a coach most needs to see. Note that the defect and the mitigation now coexist: `lib/score-display.ts` is the sanctioned path and forbids inline scaling, but these two legacy sites predate it and were not migrated.

17. Quick Reference

The same metric, calculated differently in different places

This is the most common cause of "these two screens disagree" reports. In most cases both numbers are correct — they answer different questions.

Close rate shown on...	Covers
Dashboard headline card	Whole organisation, all time, call by call
Dashboard trend / delta	Last 6 weeks maximum, weekly buckets
Team Health rows	Rep's lifetime, from cache; delta = last 7 days vs. everything before
Rep's My Page	Selected window of 2, 4 or 6 weeks, volume-weighted
AI coaching recommendations	Rep's last 20 calls
Analytics leaderboard	All loaded calls, grouped by rep name

Sample sizes behind each AI feature

Feature	Calls analysed	Refresh
Script Gap Detection	3 (full transcripts)	Every 7 days
Script Intelligence	Up to 7 (truncated to 1,500 chars)	On demand
Marketing Intelligence	3–5, random from top 10 closed	Every 7 days
Coaching recommendations	Last 20 per rep	On demand
Dashboard rubric & trends	200 most recent	Every page load
Analytics	~1,000 most recent	Every page load

Key numbers at a glance

Rule	Value
Internal score scale	0–100
Displayed score scale	0–5 (stored ÷ 20)
Green / amber / red	≥85 / 70–84 / <70
Perfect call	≥95
Overall score	Weighted average of sections (plain average if any weight is missing)
Script sections	Exactly 5, fixed names and order — convention, not enforced

Rule	Value
Section weights	Sum to 100% — checked only by the create-script form
Call outcomes	2 values: <code>closed</code> , <code>not_closed</code>
Billing rate (default)	\$0.0667 per minute
Minimum billable call	30 seconds
Billing rounding	Up, to the whole minute
Intent Index	0–5, from 4 signals scored 0–10
Default intent weights	25% each
Default billing status	<code>PAID</code> (since migration 106)

Deprecated routes

The `/overview` screen has been removed. The owner's home is `/dashboard` . Old links to `/overview` redirect there automatically rather than erroring, so bookmarks keep working.

AskMoses.AI — System Manual v1.3 · 30 September 2026 · v1.3 passages verified against `dev` @ `66cf2c6` , v1.2 passages against `18807ca` ; the rest as of `aaacf10` , 31 July 2026.